Hashing Implementation in Our Project

Introduction

In this project, we have worked with a large Spotify dataset that contains more than 10,000 songs and different types of metadata such as track names, artists, albums, and artist IDs. Since artists appear many times across different songs, we needed an efficient way to organize and retrieve songs associated with a particular artist.

To solve this, we used **hashing**. Hashing helps store data in a way that allows quick lookup. Instead of scanning the entire dataset every time we want to find songs by an artist, we can directly access them using a key.

In our implementation, the artist's name acts as the **key**, and the related song information is stored as the **value**.

Data Preprocessing

Before building the hash tables, the dataset required cleaning because real-world data often has inconsistencies.

Some of the preprocessing steps we used include:

- converting artist names to lowercase so that names like "Drake" and "drake" are treated the same
- removing extra spaces or formatting issues
- converting columns that contain multiple artists into lists
- handling missing or incorrect values in the dataset

These steps help ensure that the same artist does not appear multiple times in the hash table due to formatting inconsistencies.

Hash Table Structure

We implemented hashing using **Python dictionaries**, which internally use hash tables.
Two main dictionaries were used in our project.

1. Artist to Songs Mapping

The main dictionary stores songs for each artist.

artist_to_songs[artist_name]

Each artist maps to three categories of songs:

- solo songs
- songs where the artist is the main performer
- songs where the artist appears as a feature

For example:

artist_to_songs["travis scott"] = {
"solo": [...],
"main": [...],
"feature": [...]
}

This structure allows us to easily retrieve all songs related to an artist and understand their role in each track.

2. Artist to ID Mapping

The second dictionary stores the mapping between an artist name and their Spotify artist ID.

artist_to_id[artist_name] = artist_id

For example:

artist_to_id["travis scott"] = "0Y5tJX1MQlPlqiwlOH1tJY"

This helps us access additional artist information when needed.

Why Hashing is Useful

Hashing makes searching much faster. If we did not use hashing, we would have to go through every row in the dataset to find songs by a particular artist. That would take **O(n)** time.
With hashing, we can directly retrieve the data using the artist's name as a key. Python dictionaries usually perform lookups in **O(1)** average time.
Because our dataset contains thousands of records and many repeated artists, hashing makes the system much more efficient.

Handling Real-World Data

Since the dataset comes from real-world music data, it contains some messy or inconsistent information. For example, songs may have multiple artists or different formatting styles.

To deal with this:

- artist names are standardized before being added to the hash table
- songs with multiple artists are split so that each artist is indexed separately
- missing or invalid values are handled before inserting data

This helps keep the hash table accurate and prevents duplicate entries.

Time Complexity and Scalability

One advantage of using hashing is the improvement in lookup performance. Without hashing, searching for songs by an artist would require scanning the entire dataset, which takes O(n) time. With a hash table, the system can directly access the data using the artist name as the key. Dictionary lookups in Python usually take O(1) average time. This makes the system much faster, especially when working with large datasets like ours with more than 10,000 records. As the dataset grows, hashing allows the application to scale efficiently without significantly increasing search time.

Conclusion

Hashing is the main data structure used in our project to organize and retrieve artist-related songs. By using Python dictionaries, we were able to build a fast lookup system that groups songs by artist and allows quick search operations.

This approach works well for large datasets and helps efficiently manage music metadata from real-world sources.